EXPERIMENT – 7

Name: Probal Dhali

UID: 24BAI71013

Section/Group: 24AIT_NTPP 3 (B)

Branch/Semester: CSE_AIML (CS221) / 5th

Subject Code: 24 CSP-337

Subject Name: Full Stack - II

Date of Performance: 24/09/2026

Submitted to: Ms. Lovleen Kaur (E20454)

Experiment – 7.1

1. AIM

To implement secure backend APIs using JWT authentication and role-based authorization.

2. OBJECTIVE

- To understand JWT-based authentication.
- To configure Spring Security for API protection.
- To implement authentication filters.
- To enforce role-based access control (RBAC).

3. SOFTWARE REQUIREMENT

- Java JDK 17 or above
- Spring Boot
- Spring Security
- Postman / API testing tool
- IDE such as IntelliJ IDEA or Eclipse

4. THEORY

JWT (JSON Web Token) is a token-based authentication mechanism used to securely verify users without maintaining server-side sessions. After successful authentication, the server generates a JWT, which the client sends with subsequent requests.

Spring Security provides a security filter chain that intercepts incoming requests, validates authentication tokens, and applies authorization rules.

Probal Dhali_24BAi71013

Role-Based Access Control (RBAC) restricts access to resources according to user roles such as ADMIN and USER. Method-level security can be implemented using annotations such as `@PreAuthorize` to provide fine-grained access control.

JWT and RBAC together provide:

- Stateless authentication
- Secure API access
- Role-based authorization
- Scalable backend security

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot project with Spring Security dependencies.
2. Configure Spring Security and define protected endpoints.
3. Create a JWT authentication filter to read and validate the token.
4. Authenticate the user and generate a JWT token.
5. Add the JWT token to the `Authorization` header of API requests.
6. Define roles such as `USER` and `ADMIN`.
7. Use `@PreAuthorize` to restrict specific methods according to user roles.
8. Test authenticated and unauthorized requests using Postman.

6. CODE

Security Configuration

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
   @Bean
   public SecurityFilterChain securityFilterChain(HttpSecurity http)
         throws Exception {
      http
         .csrf(csrf -> csrf.disable())
         .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated()
         );
      return http.build();
```

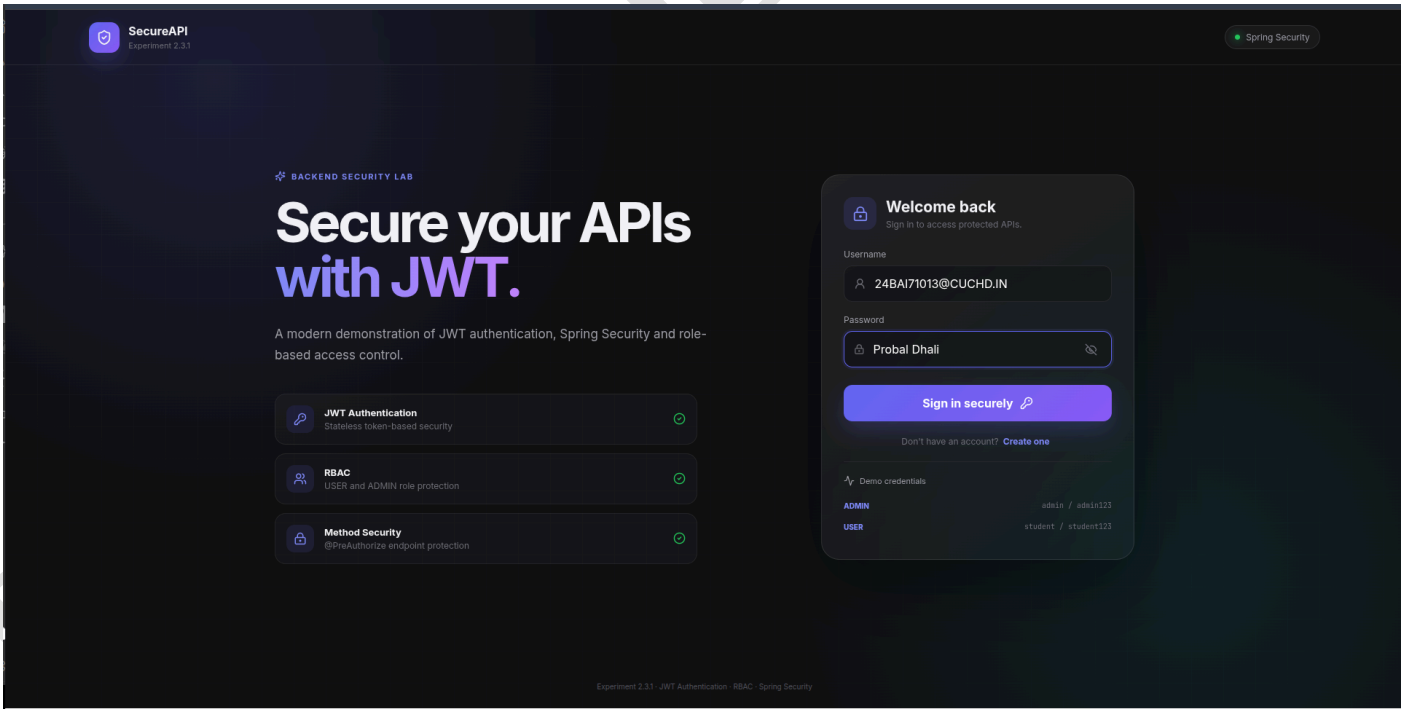
Probal Dhali_24BAi7013

```
    }
}
```

Controller with Role-Based Authorization

```
@RestController
@RequestMapping("/api")
public class TestController {
    @GetMapping("/user")
    @PreAuthorize("hasRole('USER')")
    public String userAccess() {
        return "Welcome USER";
    }
    @GetMapping("/admin")
    @PreAuthorize("hasRole('ADMIN')")
    public String adminAccess() {
        return "Welcome ADMIN";
    }
}
```

7. OUTPUT



24BAI71013

8. LEARNING OUTCOME

After completing this experiment, the student will be able to:

- Understand JWT-based authentication.
- Configure Spring Security for backend APIs.
- Implement authentication filters.
- Apply Role-Based Access Control (RBAC).
- Secure API endpoints using `@PreAuthorize`.

Probal Dhali_24BAi7013

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 7.2

1. AIM

To implement encryption and secure storage mechanisms for protecting sensitive data and credentials.

2. OBJECTIVE

- To understand encryption techniques.
- To implement AES-based data encryption.
- To securely store third-party credentials.
- To manage token lifecycle using access and refresh tokens.

3. SOFTWARE REQUIREMENT

- Java JDK 17 or above
- Spring Boot
- Java Cryptography Architecture (JCA)
- Database such as MySQL
- IDE such as IntelliJ IDEA or Eclipse

4. THEORY

Encryption is the process of converting sensitive data into an unreadable format to prevent unauthorized access. AES (Advanced Encryption Standard) is a symmetric encryption algorithm commonly used for protecting sensitive information.

In backend applications, OAuth credentials and tokens should not be stored in plaintext. AES encryption can be used to encrypt sensitive data before storing it in a database and decrypt it only when required.

Token lifecycle management commonly uses two types of tokens:

- Access Token: Short-lived token used to access protected APIs.
- Refresh Token: Long-lived token used to obtain a new access token after the current access token expires.

Using short-lived access tokens and refresh-token rotation improves security and limits the impact of token theft.

5. IMPLEMENTATION/PROCEDURE

1. Create a Spring Boot application.

Probal Dhali_24BAi7103

2. Generate a secure AES encryption key.
3. Encrypt sensitive credentials before storing them in the database.
4. Decrypt the credentials only when required.
5. Generate a short-lived access token after authentication.
6. Generate a long-lived refresh token.
7. Use the refresh token to obtain a new access token.
8. Rotate or invalidate refresh tokens when necessary.
9. Test encryption and token operations using Postman.

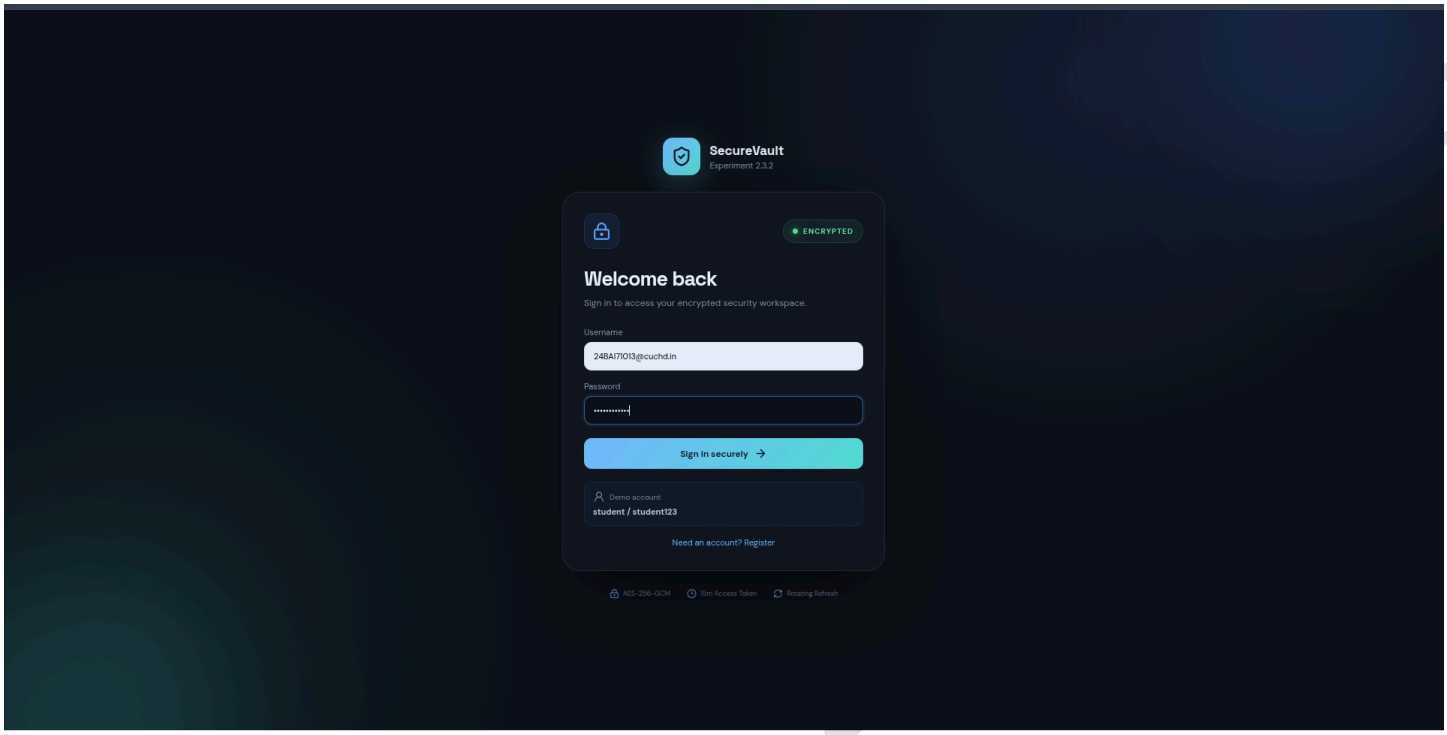
6. CODE

AES Encryption Example

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
public class AESEncryption {
   private static final String ALGORITHM = "AES";
   private static final String SECRET_KEY = "1234567890123456";
   public static String encrypt(String data) throws Exception {
      SecretKeySpec key =
            new SecretKeySpec(SECRET_KEY.getBytes(), ALGORITHM);
      Cipher cipher = Cipher.getInstance(ALGORITHM);
      cipher.init(Cipher.ENCRYPT_MODE, key);
      byte[] encrypted = cipher.doFinal(data.getBytes());
      return Base64.getEncoder().encodeToString(encrypted);
   }
   public static String decrypt(String encryptedData) throws Exception {
      SecretKeySpec key =
            new SecretKeySpec(SECRET_KEY.getBytes(), ALGORITHM);
      Cipher cipher = Cipher.getInstance(ALGORITHM);
      cipher.init(Cipher.DECRYPT_MODE, key);
      byte[] decrypted = cipher.doFinal(
         Base64.getDecoder().decode(encryptedData)
      );
      return new String(decrypted);
   }
}
```

Probal Dhali_24BAi7103

7. OUTPUT



8. LEARNING OUTCOME

After completing this experiment, the student will be able to:

- Understand encryption and data confidentiality.
- Implement AES-based encryption and decryption.
- Securely store sensitive credentials.
- Understand access and refresh token mechanisms.
- Implement secure token lifecycle management.

Probal Dhali